

Coding and Solution

Date	28 September 2026
Team ID	6ab9f220c23107aaaa73f984
Project Name	ComicCraft AI - AI comic story creator

Coding and Solution

Selected excerpts from the actual source code. The full code is in the 'project source-code' folder.

Data models (app/layout_builder.py)

```
class StoryRequest(BaseModel):
    """Input sent by the user to create a new comic."""

    prompt: str = ""
    character: str = ""
    setting: str = ""
    tone: str = "Funny"
    art_style: str = "Classic Comic Book"

class PanelData(BaseModel):
    """Data for one comic panel."""

    panel_number: int = 1

    # Short title displayed directly below the image.
    panel_heading: str = ""

    # Main story/narration shown below the heading.
    scene_description: str = ""

    # Additional visual information used when building image prompts.
    character_action: str = ""
    character_expression: str = ""

    # Dialogue is rendered as a speech bubble by the frontend.
    dialogue: str = ""

    # Short dramatic caption displayed at the bottom.
    caption: str = ""

    # Prompt sent to Stable Diffusion XL.
    image_prompt: str = ""

    # Relative URL of the generated panel image.
    image_url: Optional[str] = None

    # True when image generation failed.
    failed: bool = False

class ComicData(BaseModel):
    """Complete generated comic."""

    title: str = ""
    panels: List[PanelData] = Field(default_factory=list)
    tone: str = "Funny"
    art_style: str = "Classic Comic Book"

class ImageGenRequest(BaseModel):
    """Request used by the image-generation endpoint."""

    image_prompt: str = ""
    art_style: str = "Classic Comic Book"
    panel_number: int = 1
    comic_id: Optional[str] = "default"
```

```
class PdfExportRequest(BaseModel):
    """Request used by the PDF export endpoint."""

    title: str = ""
    panels: List[PanelData] = Field(default_factory=list)
```

API endpoint: generate comic (app/routes.py)

```
try:
    comic = generate_comic_script(request)
    return comic.model_dump()
except Exception as e:
    logger.error(f"Error generating comic: {e}", exc_info=True)
    raise HTTPException(status_code=500, detail=str(e))

@router.post("/generate-image")
async def generate_image_endpoint(req: ImageGenRequest):
    """
    Generates one panel image using Hugging Face Inference Providers.
    Returns image URL.
    """
```

API endpoint: generate panel image (app/routes.py)

```
    image_url = generate_panel_image(
        image_prompt=req.image_prompt,
        art_style=req.art_style,
        panel_number=req.panel_number
    )
    return {"image_url": image_url}
except Exception as e:
    logger.error(f"Error generating panel image: {e}", exc_info=True)
    raise HTTPException(status_code=500, detail=str(e))

@router.post("/download-pdf")
async def download_pdf_endpoint(req: PdfExportRequest):
    """
    Builds the ReportLab PDF for the generated comic.
    Returns the downloadable URL.
    """
```

Style presets (app/image_generator.py)

```
STYLE_PRESETS = {
    "Classic Comic Book": "comic-book",
    "Manga": "anime",
    "Watercolor": "digital-art",
    "Pixel Art": "pixel-art",
    "Noir": "cinematic",
    "Cartoon": "digital-art",
    "Retro Pop Art": "comic-book",
}
```